Curso 2026

Programación en Entornos Distribuidos

Grado en Ingeniería en Tecnologías para la Animación y los Videojuegos

Guion de Prácticas

Programación en Entornos Distribuidos

Grado en Ingeniería en Tecnologías para la Animación y los Videojuegos

Año académico: 2026

Dune: Arrakis Dominion Distributed

Simulación distribuida en C# sobre logística, criaturas y control imperial de la especia

Entrega obligatoria: Sí.

Lenguaje de implementación: C# sobre .NET.

Asignatura: Programación en Entornos Distribuidos.

> Esta versión revisada del enunciado adapta la práctica a C#, corrige la maquetación para integrar material visual de forma más robusta y amplía el documento con una sección teórica de fundamentos de sistemas distribuidos. El objetivo es que el estudiante no sólo implemente una aplicación ambientada en el universo de Dune, sino que también comprenda el marco conceptual que justifica sus decisiones de arquitectura, concurrencia, comunicación, sincronización, seguridad y consistencia.

La práctica es de entrega obligatoria. La fecha límite general se fija en este enunciado y no se aceptarán entregas posteriores sin autorización expresa. El desarrollo deberá realizarse en C# dentro del ecosistema .NET, utilizando una organización del código coherente con el enfoque de aplicaciones distribuidas recomendado por la documentación técnica contemporánea [4] [7].

Todo el trabajo deberá mantenerse en un repositorio GitHub privado, compartido con el profesor desde el inicio del desarrollo. El historial del repositorio deberá reflejar una evolución real del sistema y no una subida final del código en bloque.

1. Presentación y objetivos de aprendizaje

La presente práctica tiene como finalidad consolidar competencias de programación orientada a objetos, estructuras de datos, persistencia, manejo de excepciones, modelado de dominio y, de manera específica, diseño de software para

entornos distribuidos. Para ello, el alumnado desarrollará un prototipo jugable y administrable en C#, inspirado en el universo narrativo y visual de Dune, en el que distintos componentes cooperarán para sostener una simulación persistente.

La ambientación se apoya en elementos reconocibles de la franquicia. Arrakis es el planeta desértico central y la única fuente natural de melange, la especia más valiosa del Imperio 1. Organizaciones como CHOAM representan el poder económico y logístico del universo 2, mientras que mundos como Caladan aportan contraste ambiental, político y estético 3. Este contexto es idóneo para una práctica de sistemas distribuidos porque combina recursos escasos, operaciones remotas, múltiples enclaves y necesidad de coordinación entre subsistemas separados.

Desde el punto de vista técnico, la documentación de Microsoft recuerda que, cuando una aplicación crece, conviene separar responsabilidades en capas lógicas y distinguirlas del despliegue físico por procesos o nodos, algo especialmente relevante en aplicaciones distribuidas 4. Del mismo modo, Microsoft subraya que una arquitectura distribuida moderna debe prestar especial atención a los mecanismos de comunicación síncrona y asíncrona, evitando dependencias frágiles entre servicios 6.

> En términos arquitectónicos, una solución puede poseer varias capas lógicas y, al mismo tiempo, desplegarse en uno o varios tiers o nodos físicos, de manera que organización interna y topología de ejecución no son exactamente la misma cuestión 4.

Para la realización de esta práctica será necesario configurar un entorno de desarrollo con Visual Studio o Visual Studio Code y .NET SDK, crear una solución bien estructurada, documentar las decisiones de diseño y realizar commits frecuentes con incrementos funcionales verificables.

2. Fundamentos de sistemas distribuidos

Antes de abordar el enunciado práctico, el estudiante deberá comprender los principios que justifican la arquitectura propuesta. Esta sección forma parte del marco docente de la práctica y debe ser tenida en cuenta tanto para la implementación como para la memoria final.

2.1. Introducción a los sistemas distribuidos

La computación distribuida utiliza múltiples recursos computacionales situados en distintas ubicaciones operativas para alcanzar un propósito de cálculo común 5. A su vez, una definición académica ampliamente aceptada entiende un sistema distribuido como un conjunto de elementos computacionales autónomos que se presenta ante sus usuarios como un sistema coherente único 8. En términos prácticos, esto significa que diferentes procesos, máquinas o servicios cooperan y se coordinan para ofrecer una funcionalidad integrada.

Las ventajas de este enfoque son notables. En primer lugar, permite escalar el sistema mediante la incorporación de nuevas máquinas. En segundo lugar, favorece la tolerancia a fallos gracias a la redundancia. En tercer lugar, facilita la compartición de recursos, mejora el reparto de carga y permite absorber picos de demanda con mayor flexibilidad 5. Además, resulta especialmente adecuado cuando la computación debe ejecutarse cerca de los datos o de los usuarios.

Las desventajas también son relevantes y condicionan el diseño del software. Un sistema distribuido introduce latencia de red, complejidad de coordinación, problemas de sincronización, riesgo de inconsistencias temporales, fallos parciales difíciles de detectar y mayores exigencias en materia de seguridad 5. En consecuencia, programar en entornos distribuidos exige asumir que la red falla, los tiempos no son instantáneos y dos partes del sistema pueden observar estados distintos durante intervalos limitados.

Entre los ejemplos de sistemas distribuidos pueden citarse Internet, plataformas de videojuegos en línea, servicios en la nube, sistemas de bases de datos replicadas, aplicaciones basadas en microservicios, plataformas de streaming, redes peer-to-peer y servicios empresariales desplegados en múltiples servidores 5.

2.2. Modelos de programación distribuida

Los sistemas distribuidos pueden organizarse conforme a distintos modelos de interacción. Una primera distinción fundamental separa interacciones síncronas, en las que un componente solicita una operación y espera respuesta, de interacciones asíncronas, en las que el emisor no bloquea su progreso esperando inmediatamente el resultado 6. Esta distinción condiciona la latencia percibida, el acoplamiento, la resiliencia y la complejidad del sistema.

El modelo cliente-servidor es uno de los esquemas más extendidos. En él, un cliente remite peticiones a un servidor, que procesa la solicitud y devuelve una respuesta. Este modelo es adecuado para consultas, autenticación, operaciones de administración y servicios expuestos mediante API 5. En la práctica propuesta, el centro de mando puede actuar como cliente de un servicio de simulación o de persistencia.

El modelo peer-to-peer distribuye las responsabilidades entre nodos que pueden actuar simultáneamente como clientes y servidores. Este enfoque reduce la dependencia de un nodo central y resulta apropiado para redes de compartición de recursos, ciertas aplicaciones descentralizadas y algunos entornos colaborativos 5. Aunque no es obligatorio implementarlo en esta práctica, sí es útil comprenderlo como alternativa conceptual.

Los sistemas basados en eventos constituyen otra forma esencial de programación distribuida. En ellos, un componente publica un evento y otros componentes reaccionan cuando lo reciben. Microsoft destaca que los sistemas distribuidos modernos recurren con frecuencia a la comunicación orientada a eventos y a los esquemas de publish/subscribe para desacoplar servicios y favorecer su autonomía 6. Este enfoque es especialmente valioso cuando una acción de un subsistema debe propagar cambios a otros sin crear cadenas de dependencias síncronas.

2.3. Conceptos de concurrencia

La concurrencia es inseparable de los sistemas distribuidos. Un sistema concurrente permite que varios procesos, tareas o hilos progresen de forma aparentemente simultánea. IBM recuerda que la concurrencia posibilita la ejecución paralela de múltiples secuencias de instrucciones, aunque también puede generar cuellos de botella y sobrecarga si no se administra correctamente 5.

Un proceso puede entenderse como una instancia de programa en ejecución con su propio espacio de memoria. Un hilo es una unidad de ejecución más ligera que comparte memoria con otros hilos del mismo proceso. En aplicaciones distribuidas escritas en C#, ambos conceptos aparecen con frecuencia cuando se ejecutan servicios, tareas asíncronas o componentes que atienden múltiples solicitudes.

La exclusión mutua es necesaria cuando varios flujos de ejecución acceden a recursos compartidos y se desea evitar condiciones de carrera. La sincronización permite coordinar el orden de determinadas operaciones, asegurando que ciertas acciones no ocurran antes de que otras hayan concluido. En un sistema distribuido, la sincronización no se limita a hilos locales, sino que se extiende al intercambio de mensajes, al acceso a datos remotos y a la coordinación temporal entre nodos.
El interbloqueo o deadlock aparece cuando dos o más procesos quedan bloqueados esperando recursos retenidos por otros. En sistemas distribuidos, su análisis es más complejo porque la espera puede repartirse entre varios nodos, colas o servicios. Por ello, el diseño debe minimizar dependencias circulares y establecer políticas claras de tiempo de espera, reintento o cancelación.

2.4. Comunicación en sistemas distribuidos

La comunicación entre componentes distribuidos se apoya en protocolos de comunicación entre procesos, como HTTP, AMQP o protocolos binarios sobre TCP, según la naturaleza del sistema 6. La selección del protocolo influye sobre el rendimiento, la interoperabilidad, la observabilidad y el grado de acoplamiento.

Una técnica clásica es RPC (Remote Procedure Call), que permite invocar operaciones remotas con una semántica parecida a la llamada a funciones locales. Su gran ventaja es la simplicidad conceptual para el programador, aunque puede inducir a ignorar las diferencias reales entre llamada local y llamada remota, como la latencia, el fallo parcial o la

serialización de argumentos.

En entornos orientados a objetos distribuidos también aparece el concepto de RMI (Remote Method Invocation), cercano a RPC pero enfocado a la invocación de métodos sobre objetos remotos. Aunque en esta práctica no es obligatorio implementar RMI en sentido estricto, sí resulta útil comprender la idea de que ciertos objetos o servicios pueden estar ubicados fuera del proceso local.

Los sistemas de mensajes constituyen otra base esencial. Microsoft destaca que la mensajería asíncrona permite que un emisor publique una orden o un evento sin esperar respuesta inmediata, favoreciendo la autonomía y resiliencia de los servicios 6. Este tipo de comunicación es especialmente recomendable para notificaciones, propagación de cambios, registro de eventos y actualización de estados entre componentes desacoplados.

2.5. Sincronización en sistemas distribuidos

La sincronización en sistemas distribuidos excede el mero uso de cerrojos o semáforos. Un problema fundamental es la sincronización de relojes, porque los nodos distribuidos no comparten una fuente temporal perfecta y pueden observar tiempos diferentes. Esto afecta al orden de eventos, a la resolución de conflictos y a la auditoría de operaciones 8.

Los algoritmos de sincronización intentan aproximar una noción temporal común o, al menos, ordenar correctamente los eventos relevantes. En la práctica, esto influye sobre registros de auditoría, marcas de tiempo, caducidad de mensajes y análisis de trazas. En aplicaciones reales suelen combinarse protocolos de sincronización temporal con estrategias lógicas de ordenación de eventos.

También es importante la sincronización de bases de datos. Cuando un sistema distribuido mantiene información repartida o replicada, debe establecer cómo y cuándo se propagan los cambios. Microsoft señala que, en lugar de forzar transacciones distribuidas fuertes entre servicios, muchas arquitecturas modernas adoptan consistencia eventual y propagan cambios mediante eventos o mensajería 7.

2.6. Seguridad en sistemas distribuidos

La seguridad en sistemas distribuidos es más exigente que en sistemas centralizados porque existen más puntos de entrada, más canales de comunicación y más superficies de ataque. Entre las amenazas principales se encuentran la interceptación de tráfico, la suplantación de identidad, la manipulación de mensajes, el acceso no autorizado a recursos, la exposición indebida de datos y los ataques de denegación de servicio.
Las políticas y mecanismos de seguridad deben abarcar autenticación, autorización, control de acceso, cifrado de comunicaciones, gestión de secretos, validación de mensajes, auditoría y trazabilidad. NIST subraya la importancia del control de acceso coordinado en sistemas distribuidos emergentes, destacando que estos entornos requieren capacidades de seguridad comparables a las de los sistemas no distribuidos, pero adaptadas a su mayor heterogeneidad 9.

Los sistemas criptográficos desempeñan un papel esencial, ya que permiten garantizar confidencialidad, integridad y autenticidad. En la práctica, esto se traduce en el uso de canales seguros, firmas o verificaciones de integridad, almacenamiento responsable de credenciales y protección de datos sensibles.

La seguridad en la red incluye segmentación, monitorización, protección del canal de transporte y tratamiento defensivo de errores. Para esta práctica, se espera al menos una reflexión explícita sobre validación de entradas, protección de ficheros persistentes, separación de responsabilidades y tratamiento seguro de las comunicaciones entre componentes.

2.7. Consistencia y replicación

La consistencia de datos describe el grado en que los distintos componentes observan un estado compatible del sistema. En una aplicación monolítica es habitual apoyarse en transacciones ACID fuertes, pero en arquitecturas distribuidas esa opción puede volverse costosa o inviable 7. Por ello, muchos sistemas priorizan disponibilidad y escalabilidad, aceptando formas de consistencia eventual.

La coherencia de caché aparece cuando varias copias temporales de los datos deben mantenerse razonablemente alineadas. En contextos distribuidos, una caché obsoleta puede provocar decisiones erróneas, por lo que el sistema debe definir políticas de invalidación, expiración y actualización.

Los protocolos de replicación permiten mantener múltiples copias de datos o servicios para mejorar disponibilidad, rendimiento y tolerancia a fallos. Microsoft indica que, cuando un servicio necesita que otro refleje ciertos cambios, una estrategia recomendable consiste en propagar únicamente la información necesaria mediante eventos e integración asíncrona 7. De este modo, cada componente conserva soberanía sobre sus datos, pero la solución global mantiene coherencia funcional suficiente.

3. Enunciado práctico

Como parte de su formación en el Grado en Ingeniería en Tecnologías para la Animación y los Videojuegos, el estudiante se incorporará a un estudio ficticio denominado CHOAM Distributed Entertainment, especializado en simuladores estratégicos y mundos persistentes. La empresa desea lanzar una experiencia de gestión ambientada en Dune, pero no quiere un videojuego monolítico, sino un prototipo técnicamente planteado para funcionar en un contexto distribuido.

El proyecto recibe el nombre de Dune: Arrakis Dominion Distributed. En él, el jugador gestionará una Casa Menor autorizada a operar enclaves de explotación, investigación y exhibición relacionados con la especia, la fauna y el equilibrio económico del desierto. La aplicación deberá ofrecer una simulación de rondas, persistencia de estado, trazabilidad de acciones y una arquitectura preparada para evolucionar hacia varios servicios cooperantes.

La inspiración visual del proyecto se apoya en la iconografía desértica de Arrakis y en el contraste con otros mundos del universo Dune, como Caladan 1. A continuación se incluyen imágenes de referencia que deberán visualizarse en la versión final del documento.

Figura 1. Referencia visual de Arrakis y del gusano de arena.

Figura 2. Referencia visual de Caladan para contrastar biomas y escenarios.

3.1. Requisitos funcionales

Los creativos del estudio han concebido un juego en el que el jugador se pone al frente de una Casa Menor autorizada a operar en distintos enclaves del Imperio. El objetivo no consiste únicamente en acumular riqueza, sino en demostrar que la Casa puede mantener una operación estable y rentable en un entorno hostil, gestionando recursos, visitantes, criaturas y recintos especializados.

Cada partida tendrá un identificador único y estará compuesta por varios componentes de información distribuidos. Aunque el estudiante podrá decidir el diseño final, la práctica deberá permitir distinguir, como mínimo, entre un cliente de administración, un módulo o servicio de simulación, un módulo o servicio de persistencia y un modelo compartido de dominio. Esta separación podrá materializarse como varios proyectos dentro de una misma solución, como varios procesos ejecutables comunicados entre sí o como una combinación razonada de ambas aproximaciones 4.

Elemento	Descripción funcional
Alias del jugador	Identificador único del jugador dentro del sistema.
Partida	Entidad principal, identificada unívocamente y almacenable para reanudación posterior.
Enclave de aclimatación	Área destinada a la cría, mantenimiento y evolución inicial de criaturas.
Enclave de exhibición	Área pública destinada a la exhibición y monetización de criaturas.

Fondos	Cantidad de solaris disponibles para compras, traslados y ampliaciones.
Centro de mando	Interfaz desde la que se consulta y gobierna el estado distribuido de la partida.
Servicio de simulación	Componente encargado de ejecutar las rondas mensuales y recalcular estados.
Servicio de persistencia	Componente encargado de guardar y cargar partidas y configuraciones.
Registro de eventos	Histórico de acciones de usuario y sucesos internos del sistema.

3.2. Escenarios del juego

Las partidas se desarrollarán en un escenario que fijará las condiciones iniciales del jugador. Cada escenario establecerá el número de solaris iniciales, el enclave de exhibición principal y parte del tono estratégico de la partida. El enclave de aclimatación será común y estará ubicado en una zona experimental de Arrakis controlada por la Casa del jugador.

Escenario	Descripción temática	Solaris iniciales	Enclave de exhibición
Arrakeen: Dominio de la Especia	Escenario situado en Arrakis, planeta desértico y única fuente natural de melange 1. Operación prestigiosa, compleja y altamente rentable.		100000 Arrakeen
Giedi Prime: Galería Industrial	Escenario de alta afluencia y baja exclusividad, marcado por una estética industrial y una operación compacta.	50000	Giedi Prime
Caladan: Reserva Ducal	Escenario asociado al mundo oceánico históricamente vinculado a la Casa Atreides 3. Ofrece mejores condiciones logísticas y mayor inversión inicial.	150000	Caladan

3.3. Enclaves

Los enclaves determinan la cantidad de espacio y recursos disponibles para las actividades de la Casa. Cada enclave posee hectáreas útiles, instalaciones, población de criaturas y un almacén general de suministros. Los enclaves de exhibición añaden además el número de visitantes mensuales y el nivel adquisitivo del público, que podrá ser ALTO, MEDIO o BAJO.

Enclave	Actividad	Suministros iniciales	Hectáreas	Visitantes	Nivel adquisitivo
Cuenca Experimental de Arrakis	ACLIMATACIÓN	20000	5000		—
Arrakeen	EXHIBICIÓN	10000	7700		ALTO

Giedi Prime	EXHIBICIÓN	5000	100	2000	BAJO
Caladan	EXHIBICIÓN	25000	10000	3000	MEDIO

La unidad de suministro tendrá un coste fijo de 5 solaris. Cada enclave podrá almacenar, como máximo, un número de unidades equivalente al triple de sus hectáreas. El movimiento de suministros desde el almacén general a una instalación concreta no tendrá coste adicional, pero ninguna instalación podrá superar en reservas el valor numérico de su coste de construcción.

En los enclaves de exhibición, la evolución de visitantes se calculará manteniendo el espíritu algorítmico de la práctica original.

Visitantes que llegan en un mes concreto

$$visitantesLlegan = \left(\frac{visitantesMesEnclave \times hectareasInstalaciones}{hectareasEnclave}\right) \times \frac{saludMediaCriaturas}{100}$$

Visitantes que abandonan el enclave

$$visitantesAbandonan = visitantesEnclave - \left(\frac{visitantesEnclave \times hectareasInstalaciones}{hectareasEnclave}\right) \times \frac{saludMediaCriaturas}{100}$$

3.4. Instalaciones

En los enclaves se podrán construir instalaciones de aclimatación y de exhibición. Cada instalación tendrá un coste de construcción, ocupará una determinada extensión y estará preparada para criaturas de un medio concreto y de un patrón de alimentación específico.

Tipo	Código	Coste	Medio	Alimentación	Suministros iniciales	Capacidad		Tipo de recinto
Aclimatación		1000	DESIERTO	RECOLECTOR	200	5	10	ROCA SELLADA
Aclimatación		2500	DESIERTO	DEPREDADOR	300	3	50	ESCUDO ESTÁTICO
Aclimatación			AÉREO	DEPREDADOR	500	2	100	CÚPULA BLINDADA
Aclimatación		3500		O	100	4	25	POZO REFORZADO
Exhibición	EDR02	21000	DESIERTO	RECOLECTOR	0	2	200	ROCA SELLADA

Exhibición	EDP03	12500	DESIERTO	DEPREDADOR	0	3	300	ESCUDO ESTÁTICO
Exhibición	EAV02		AÉREO	DEPREDADOR	0	2	200	CÚPULA BLINDADA
Exhibición	ESU03	25000		D	0	3	400	POZO REFORZADO

Las instalaciones de aclimatación ejecutarán tareas de alimentación y reproducción o clonación controlada. Si la instalación dispone de capacidad libre, se intentará generar una nueva criatura compatible. La especie concreta se seleccionará aleatoriamente y el proceso sólo tendrá éxito en el 20% de los intentos mensuales.

Las instalaciones de exhibición compartirán el protocolo de alimentación, pero añadirán la monetización derivada de los visitantes. Cada vez que un visitante elija una criatura como favorita, el jugador obtendrá una donación cuyo valor dependerá del estado del ejemplar y del nivel adquisitivo del enclave.

Donación por criatura favorita

[donacion = 10 \times \frac{saludCriatura}{100} \times \frac{edadCriatura}{edadAdulta} \times \sigma] donde

(\sigma = 1) si el nivel adquisitivo es BAJO, (\sigma = 15) si es MEDIO y (\sigma = 30) si es ALTO.

3.5. Criaturas

Cada criatura tendrá un identificador único, un medio, un patrón de alimentación, una edad actual, una edad adulta, un porcentaje de salud y un contador de veces que ha sido elegida como favorita. La versión inicial del juego incluirá, al menos, las siguientes criaturas.

Criatura	Medio	Alimentación	Edad adulta
Gusano de arena juvenil	SUBTERRÁNEO	DEPREDADOR	24
Tigre laza	DESIERTO	DEPREDADOR	38
Muad’Dib	DESIERTO	RECOLECTOR	12
Halcón del desierto	AÉREO	DEPREDADOR	16
Trucha de arena	SUBTERRÁNEO	RECOLECTOR	42

Criatura	Apetito base

Gusano de arena juvenil	5
Tigre laza	8
Muad'Dib	2
Halcón del desierto	2
Trucha de arena	10

Ingesta antes de la edad adulta

[cantidad = apetitoEspecie \times edad]

Ingesta tras la edad adulta

[cantidad = apetitoEspecie \times 2^{(edad - edadAdulta)} \times \alpha]

siendo (\alpha = 1) en exhibición y (\alpha = 15) en aclimatación.

La salud inicial de cualquier criatura será 100. Si recibe menos del 25% de la ingesta requerida, perderá 30 unidades de salud. Si recibe entre el 25% y el 75%, perderá 20. Si recibe más del 75% pero menos del 100%, perderá 10. Si es alimentada correctamente y no tiene salud completa, recuperará 5 unidades. Cuando su salud llegue a 0, la criatura entrará en letargo irreversible.

3.6. Traslado y descarte de criaturas

Las criaturas adultas podrán trasladarse desde instalaciones de aclimatación a instalaciones de exhibición. Para ello, la instalación de destino no podrá estar completa, la criatura deberá tener una salud igual o superior a 75 y el jugador tendrá que disponer de fondos suficientes para cubrir el coste del traslado.

Coste de traslado

[costeTraslado = 100 \times 3^{(edadCriatura - edadAdulta)} \times \sigma]

donde (\sigma = 5) para criaturas de DESIERTO, (\sigma = 15) para criaturas de medio AÉREO y (\sigma = 25) para criaturas de medio SUBTERRÁNEO.

Cuando sea necesario descartar una criatura, se considerará que es transferida a instalaciones reservadas de la Bene Tleilax, con un coste fijo de 20000 solaris 2.

3.7. Arquitectura distribuida mínima exigida

La solución deberá diseñarse, como mínimo, con una estructura que permita distinguir responsabilidades de forma clara. No se exige una única implementación, pero sí una organización técnica coherente.

Componente	Responsabilidad principal	Implementación orientativa
Cliente de administración	Configurar partida, consultar estado y lanzar operaciones	Consola, escritorio o interfaz ligera en C#

Servicio de simulación	Ejecutar rondas mensuales y recalcular estados	Proyecto independiente en .NET
Servicio de persistencia	Guardar y cargar partidas	JSON, serialización o almacenamiento estructurado
Modelo compartido	Entidades, contratos, enumerados y DTOs	Biblioteca de clases en C#
Registro de eventos	Historial y auditoría técnica	Ficheros, base local o bitácora estructurada

Se valorará especialmente que la solución explicite el tipo de comunicación utilizado entre componentes, la estrategia de serialización y el tratamiento de errores de comunicación.

3.8. Centro de mando

El centro de mando es imprescindible para que el jugador pueda consultar el estado completo del sistema. Deberá mostrar instalaciones, atributos, criaturas alojadas, fondos, recursos, visitantes y eventos. Las criaturas deberán aparecer ordenadas de forma decreciente por salud y los eventos deberán mostrarse cronológicamente.

En una solución distribuida, el centro de mando integrará información procedente de más de un componente. Por ello, se valorará especialmente que el estudiante diseñe una interfaz clara para consultar estado agregado, confirmaciones de guardado, advertencias y errores de sincronización.

3.9. Gestión de errores y robustez

Además del cumplimiento funcional, el software deberá incorporar controles de integridad y gestión de excepciones adecuados. Será obligatorio validar operaciones de compra sin saldo, movimientos imposibles, intentos de traslado inválidos, estados corruptos de carga, errores de comunicación entre componentes y cualquier incoherencia entre criaturas e instalaciones.

Dado que la asignatura está centrada en entornos distribuidos, se recomienda además gestionar explícitamente errores de serialización, indisponibilidad temporal de servicios, políticas de reintento, validación de mensajes entrantes y trazabilidad de fallos.

3.10. Persistencia e interfaz

Todos los datos del juego deberán almacenarse de forma persistente. La aplicación tendrá que guardar la partida cuando el usuario lo solicite y, en todo caso, al salir. También deberá poder cargar sesiones anteriores al arrancar o bajo petición. Para la asignatura se recomienda utilizar formatos interoperables como JSON.

La aplicación deberá disponer de una interfaz que permita configurar la partida, ejecutar rondas y guardar el estado antes del comienzo de una nueva ronda. Si se implementa una interfaz textual, la práctica será valorada sobre 10 puntos. Si se implementa una interfaz gráfica en tecnologías del ecosistema .NET, como Windows Forms, WPF o equivalentes justificadas para 2026, el estudiante podrá aspirar hasta a dos puntos extra.

4. Entregas reorganizadas

Con el fin de adaptar mejor la práctica a la asignatura Programación en Entornos Distribuidos, el calendario de entregas se reorganiza en cuatro hitos progresivos. Cada hito responde a una capa de madurez diferente del sistema y busca evitar que toda la carga de trabajo se concentre al final.

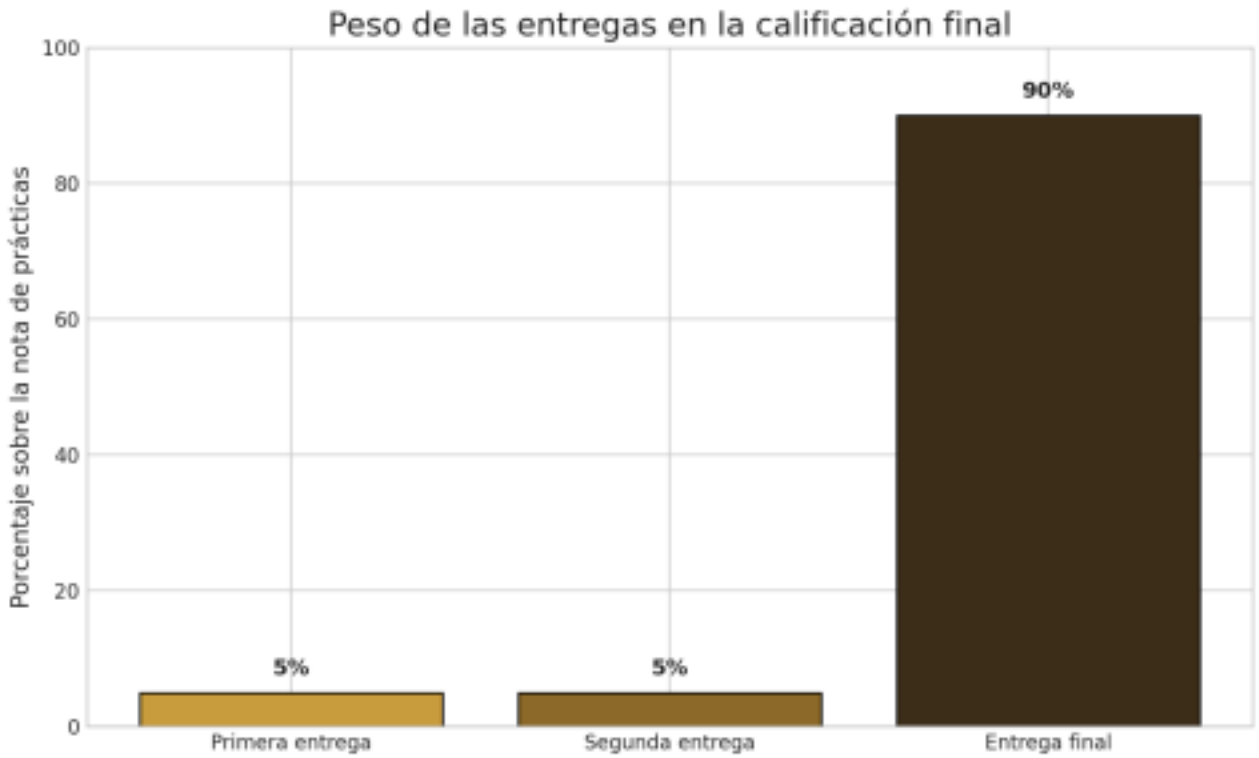

Figura 3. Peso relativo de las entregas de la práctica.

Figura 4. Visualización general del calendario de trabajo.

4.1. Entrega 1 — Diseño de dominio y arquitectura distribuida

Fecha recomendada: 17 de abril de 2026.

Peso: 10%.

En esta primera entrega el estudiante deberá presentar el modelo de dominio, la estructura inicial de la solución, los proyectos previstos, la definición de entidades, enumerados, DTOs y una justificación preliminar de la arquitectura distribuida. También deberá especificar qué componentes existirán, cómo se comunicarán y qué responsabilidades asumirá cada uno.

4.2. Entrega 2 — Comunicación, persistencia y prototipo funcional mínimo

Fecha recomendada: 8 de mayo de 2026.

Peso: 20%.

En la segunda entrega deberá existir un sistema mínimamente ejecutable que permita crear partida, almacenar estado y recuperar información. Se espera ya una primera implementación del cliente de administración, del servicio o módulo de persistencia y de un mecanismo de comunicación entre componentes, aunque sea básico.

4.3. Entrega 3 — Simulación, concurrencia y consistencia

Fecha recomendada: 22 de mayo de 2026.

Peso: 20%.

La tercera entrega deberá incorporar la lógica principal de simulación, la ejecución de rondas, la gestión de criaturas, visitantes y recursos, así como una primera resolución de los problemas de concurrencia, sincronización y consistencia de estado. En esta fase se valorará especialmente la calidad del registro de eventos y el tratamiento razonable de fallos.

4.4. Entrega final — Sistema completo y memoria técnica

Fecha límite: 3 de junio de 2026.

Peso: 50%.

La entrega final incluirá la solución completa, completamente compilable, junto con una memoria técnica del trabajo realizado. La memoria deberá contener, como mínimo, los apartados recogidos en la tabla siguiente.

Apartado de la memoria	Contenido mínimo esperado
Análisis de la aplicación	Organización general de la solución y objetivos funcionales.
Fundamentos distribuidos aplicados	Relación entre teoría y decisiones adoptadas en la práctica.
Diseño de arquitectura	Proyectos, servicios, contratos y comunicación entre componentes.
Concurrencia y sincronización	Estrategias utilizadas para coordinar procesos, tareas o recursos compartidos.
Persistencia y consistencia	Estrategia de guardado, recuperación, serialización y coherencia entre componentes.
Seguridad	Medidas básicas de validación, protección de datos y control de errores.
Estructuras de datos	Tipos utilizados y justificación.
Ordenación y búsqueda	Técnicas implementadas y motivo de elección.
UML o diagramas equivalentes	Representación estructural del sistema.
Fallos conocidos	Funcionalidades no implementadas o limitaciones.
Conclusiones	Valoración del trabajo y tiempo dedicado.

5. Evaluación

Para superar la práctica, además del software entregado, el estudiante deberá realizar una defensa respondiendo a preguntas formuladas por el profesor sobre funcionamiento, diseño, decisiones arquitectónicas, concurrencia, comunicación, sincronización, seguridad y detalles del código. Si no superase la defensa, la nota final de la práctica será de cero puntos.

Se evaluará la organización y estructuración del código, el uso correcto de programación orientada a objetos, la elección de estructuras de datos, la aplicación de técnicas de búsqueda y ordenación, la calidad de la persistencia, la claridad del código, la calidad del diseño distribuido, la solidez del tratamiento de errores y la capacidad del estudiante para justificar técnicamente su solución.

Se utilizarán asimismo mecanismos de control académico y detección de similitud. La copia total o parcial del trabajo implicará la aplicación de las medidas correspondientes.

6. Repositorio y seguimiento del trabajo

Para que la práctica sea evaluable es obligatoria la utilización de un repositorio GitHub. En él se deberá registrar toda la actividad de desarrollo. Los commits deberán realizarse cada vez que exista un incremento funcional significativo, evitando tanto el abuso de commits vacíos como la acumulación de todo el trabajo en unos pocos envíos.

Se valorará positivamente que el historial del repositorio refleje una evolución real del sistema distribuido, con aparición progresiva de proyectos, contratos, persistencia, servicios, cliente, simulación y pruebas.

7. Tutorías

Se recomienda que el estudiante asista a reuniones periódicas con el profesor para validar la estructura del código y la arquitectura distribuida elegida. Las dudas y consultas relativas a la práctica se resolverán en horario de tutorías, previa cita por correo electrónico.

IBM define la computación distribuida como el uso de múltiples recursos computacionales situados en ubicaciones operativas diferentes para un propósito de computación común . IBM también explica que el objetivo de una red distribuida es comportarse como si fuera un único sistema coherente, lo que se logra mediante paso de mensajes y protocolos de comunicación entre componentes . Entre las ventajas señaladas destacan la compartición de recursos, la tolerancia a fallos, la escalabilidad mediante incorporación de más máquinas y la capacidad para absorber variaciones de carga . Entre los inconvenientes prácticos se encuentran la latencia, los cuellos de botella y la complejidad derivada de la concurrencia .

IBM recoge además que los modelos de arquitectura distribuida incluyen el esquema cliente-servidor y el esquema peer-to-peer, y que la coordinación entre nodos depende del tipo de acoplamiento y del intercambio de mensajes . Esto resulta útil para redactar una sección introductoria orientada a estudiantes de Programación en Entornos Distribuidos.

[1] What is distributed computing? | IBM

Notas complementarias sobre comunicación, consistencia y replicación

La documentación de Microsoft sobre microservicios subraya que una aplicación distribuida ejecutada en varios procesos o servicios debe comunicarse mediante protocolos de comunicación entre procesos, como HTTP, AMQP o protocolos binarios sobre TCP, según el escenario . También diferencia entre comunicación síncrona y asíncrona, así como entre interacciones de receptor único y de múltiples receptores, destacando que la integración asíncrona favorece la autonomía y resiliencia de los servicios . Esta base resulta adecuada para explicar en el documento los modelos de interacción, el modelo cliente-servidor, los sistemas basados en eventos y la mensajería.

Microsoft señala además que las arquitecturas distribuidas modernas tienden a evitar dependencias síncronas en cadena entre servicios y recomiendan propagar cambios mediante mensajería o eventos, especialmente cuando se pretende mantener independencia operativa entre componentes . Esto ofrece un fundamento directo para introducir RPC, sistemas de mensajes y comunicación orientada a eventos como parte del marco teórico de la práctica.

En relación con la consistencia y replicación, Microsoft explica que, cuando cada servicio posee sus propios datos, no debe acceder directamente a los datos internos de otro servicio. Para mantener la coherencia entre componentes, propone recurrir a consistencia eventual, apoyada en comunicación dirigida por eventos y mecanismos de publish/subscribe . La misma fuente remarca el compromiso clásico entre disponibilidad y consistencia fuerte, recordando que muchas arquitecturas distribuidas priorizan disponibilidad y escalabilidad, y complementan esa decisión mediante técnicas de replicación y sincronización de datos .

Estas ideas permiten justificar en la nueva sección teórica los apartados de protocolos de replicación, consistencia de datos, coherencia de caché y sincronización de bases de datos, además de reforzar la parte práctica de la asignatura orientada al diseño de soluciones distribuidas en C#.

[2] Communication in a microservice architecture - .NET | Microsoft Learn

[3] Challenges and solutions for distributed data management - .NET | Microsoft Learn